Coding Agent Best Practices

OpenClaw Sub-agent Configuration Guide

May 6, 2026

Source: OpenClaw docs + community best practices

1. Agent Architecture

A coding agent in OpenClaw is a sub-agent — a background worker spawned from the main agent. Key architectural facts:

- Sub-agents run in their own isolated session: agent:<id>:subagent:<uuid>
- They get ONLY AGENTS.md + TOOLS.md injected (no SOUL, IDENTITY, USER, HEARTBEAT) — stays focused on task
- Default context mode is "isolated" (clean transcript each spawn) — recommended for coding
- Use "fork" only when the child needs the requester's conversation context
- Completion is push-based — don't poll, just wait for the announce event
- Each sub-agent is tracked as a background task

Nesting

- Max spawn depth default: 1 (sub-agents can't spawn their own)
- Set maxSpawnDepth: 2 for orchestrator pattern (main -> orchestrator -> workers)
- Depth-2 workers are leaf nodes — cannot spawn further

2. Model & Thinking Settings

These settings determine the intelligence and cost of your coding agent:

Setting	Recommendation	Why
Model	kimi-k2.5:cloud (primary)qwen3-coder:cloud (fa	Kimi strong at coding logic;Qwen-Coder tuned for code ger
thinkingDefault	medium or high	Medium for routine tasks;high for complex architecture/deb
reasoningDefault	off	Reasoning adds latency; only enable for deep analysis
Sub-agent model	Same or cheaper	Use cheaper model for repetitive sub-tasks;keep good mod
Fast mode	false (default)	Coding needs careful thought, not speed

Model budget note

Each sub-agent has its own context and token usage. For heavy/repetitive tasks, set a cheaper model via agents.defaults.subagents.model. Keep your main agent on a higher-quality model.

3. Persistent Memory Design

Sub-agents don't retain memory between sessions natively. The standard pattern:

The Memory File Pattern (our approach)

- Create a dedicated markdown file: agents/coder-memory.md
- Sub-agent reads it FIRST every session (mandatory)
- Sub-agent updates it LAST every session (mandatory)
- Include: active projects, architecture decisions, known issues, TODO items
- Keep it structured with clear sections for fast scanning
- This is analogous to PROJECT_MEMORY.md in PLCTools Coder

What to store

- Project context and current state

- Decisions made and their rationale
- Known bugs or technical debt
- Pending work items (TODO)
- Completed work log (reverse chronological)

Why markdown files work better than 'mental notes'

- Survives session restarts
- Human-readable and human-editable
- Git-tracked — full version history
- Can be shared across agent instances
- Tokens are cheaper than context window for long-running knowledge

4. Tool Configuration

Sub-agents use the same tool policy pipeline as the parent, then get the sub-agent restriction layer applied:

Tool Category	Status	Notes
File tools (read, write, edit)	ALLOWED	Core coding tools
Shell (exec, process)	ALLOWED	Build, test, run
Web (search, fetch)	ALLOWED	Research, docs lookup
Session tools (spawn, list, history)	DENIED by default	Depth-1 orchestrators get these when maxSpawnDepth >=
System tools (gateway, cron)	DENIED	Not needed for coding
Browser	OPTIONAL	Add via tools.alsoAllow if needed for web testing

Tool profiles

- "coding" profile includes: read, write, edit, exec, process, web_search, web_fetch, sessions_spawn
- "full" profile adds: browser, canvas, cron, gateway, etc.
- Per-agent override: agents.list[].tools.alsoAllow to add specific tools

5. Timeout & Reliability

Parameter	Recommended	Why
runTimeoutSeconds	600-900 (10-15 min)	Coding tasks often take 5-10 minutes;complex ones need h
Agent timeout	600s	Default agent turn timeout
Sub-agent archive	60 min (default)	Auto-archives old sub-agent sessions
Heartbeat timeout	45s (default)	Keep short for prompt housekeeping

Why timeouts matter

Without a timeout, a stuck sub-agent can hang indefinitely. Our earlier issues with trading-arena timing out at 300s confirmed: coding/analysis agents need AT LEAST 600s. For the coder agent, 900s is safer for tasks that involve building, testing, and iterating.

6. Cron Auto-Spawn Pattern

Keep coding agents warm with staggered auto-spawn:

```
Cron: coder-auto-spawn
```

```
Schedule: "0 1,5,9,13,17,21 * * *" (every 4h, offset from PLC)
Payload: agentTurn with task to read memory and wait
Timeout: 600s
Delivery: --no-deliver (no Telegram spam)
```

Stagger from plc-coder-auto-spawn (0,4,8,12,16,20) by 1 hour to avoid resource contention.

7. Current Setup vs Best Practices

Area	Current	Best Practice	Status
Model	kimi-k2.5:cloud	kimi-k2.5 or qwen3-coder	GOOD
Memory	coder-memory.md	Markdown file, read first/update las	GOOD
Identity	Separate IDENTITY file	Don't reference being Spock	GOOD
Timeout	600s via cron	600-900s	GOOD
Context	Isolated	Isolated for fresh tasks	GOOD
Thinking	Not set explicitly	medium for coding	ADD
Fallback model	Not set	qwen3-coder:cloud	ADD

Recommended additions

- Add thinkingDefault: "medium" to coder config
- Add fallback model: ollama/qwen3-coder:cloud
- Consider tools.allow to restrict to coding-specific tools only

8. Quick Reference Card

Spawning a coding sub-agent

```
sessions_spawn(
  label: "my-task",
  task: "Read coder-memory.md first, then [task description]",
  runTimeoutSeconds: 900
)
```

Key agent config file

```
Location: agents/coder.md
Memory: agents/coder-memory.md
Registry: agents/REGISTRY.md
```

Key docs

- Local: node_modules/openclaw/docs/tools/subagents.md
- Local: node_modules/openclaw/docs/gateway/config-agents.md
- Web: https://docs.openclaw.ai/subagents
- Web: https://docs.openclaw.ai/gateway/config-agents